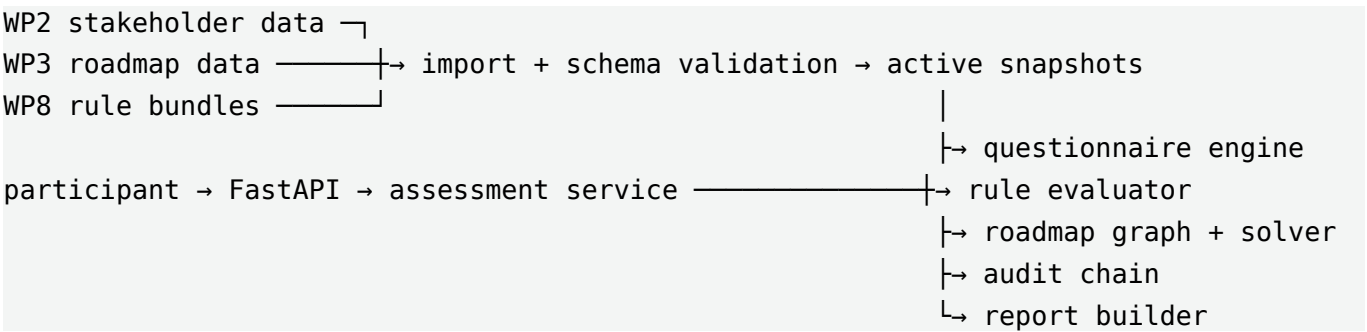


Volume 03 — System Architecture & Technical Design

1. Architecture intent

The architecture separates source ingestion, versioned evidence, deterministic domain logic, API access, and report presentation. This enables the consortium to improve upstream content without changing the core assessment flow and lets the demonstrator run with safe fixtures when partner data is incomplete.

2. Logical architecture



The authoritative V1 decision path is deterministic. AI, RAG, GraphRAG, and direct external-system actions are future extensions, not required runtime dependencies.

3. Components

Component	Responsibility
Web client	Selects profile/scenario, starts an assessment, displays evidence and report links.
FastAPI facade	Authenticates the demo request, validates payloads, and exposes versioned endpoints.
Assessment service	Orchestrates session state, imports, questionnaire processing, recommendations, and reporting.
Questionnaire engine	Loads versioned questions and derives a stakeholder state.
Rule loader/evaluator	Validates rule bundles and evaluates conditions deterministically.
Roadmap graph	Stores nodes and edges and supports current/target location and shortest-path search.
Persistence	Stores sessions and audit events when PostgreSQL is configured; in-memory state remains a

Component	Responsibility
	demo fallback.
Audit chain	Links events with previous hashes and exposes chain validity in the report.
Import adapters	Convert WP2/WP3/WP8 payloads into domain models and retain source metadata.

4. Runtime data flow

1. Startup fetches or loads approved WP2, WP3, and WP8 snapshots.
2. Each payload is validated and assigned a snapshot/version identity.
3. The active questionnaire, roadmap, and rules are constructed from those snapshots.
4. A participant creates a session bound to stakeholder type, target scenario, and active versions.
5. Answers derive a readiness state and confidence markers.
6. The solver locates the current and target roadmap nodes, evaluates rules, and searches for a path.
7. The report builder combines the result with trace and audit evidence.
8. Export creates a reviewer artifact containing the demo disclaimer and source versions.

5. Deployment baseline

V1 supports a local technical-review deployment with the FastAPI service, web client, mock WP2/WP3/WP8 services, and optional PostgreSQL persistence. Docker Compose may package these services, but the core demo must not depend on a production Kubernetes environment. A production deployment profile is future work.

6. Availability and failure modes

Failure	Expected behavior
WP2 unavailable	Use approved demo/fallback taxonomy and flag the snapshot.
WP3 labels incomplete	Continue with node IDs and missing-data warning.
WP8 rules unavailable	Return unverified compliance state; do not imply compliance.
No route exists	Return blocked with prerequisites and triggered rules.
Persistence unavailable	Continue only where the demo profile allows; surface the limitation.
Invalid import	Reject activation and preserve the previous active snapshot.

7. Audit and provenance design

Each auditable event contains an event ID, event type, timestamp, subject/session ID, actor class, summarized payload, source version, previous hash, and event hash. Reports include a chain-validity result. Sensitive values are minimized; answers and identifiers are not copied into logs beyond what the proposal contract requires.

8. Architecture decisions

ADR-01 Deterministic V1 decision engine

Use explicit questionnaire, rules, and graph search for V1. This makes acceptance repeatable and reviewable. LLM/RAG recommendation generation is deferred.

ADR-02 Version every input set

Bind sessions and reports to questionnaire, roadmap, rule, schema, and upstream snapshot versions. This is necessary to reproduce a recommendation after partner data changes.

ADR-03 Graceful degradation is explicit

Prefer a visible degraded or blocked result over silent fallback or fabricated completeness.

ADR-04 Keep data contracts portable

Use JSON/YAML/JSON Schema/OpenAPI as exchange formats. PostgreSQL or another graph backend can change behind the domain interfaces without changing the proposal contract.

9. Architecture verification

The architecture is verified by the end-to-end demo: load fixtures, create a session, submit or derive state, generate a recommendation, inspect trace fields and audit validity, and export the report. The detailed evidence plan is in Volume 07.